

TECHNISCHE UNIVERSITÄT DORTMUND
FAKULTÄT STATISTIK

FALLSTUDIEN II / CASE STUDIES II
WINTER SEMESTER 2025/26

SPOTIFY RECOMMENDATION ALGORITHM ANALYSIS

GROUP 4

JIAHUI FAN, ÖMERCAN MISIRLIOĞLU, YORDAN SAPUTRA

SUPERVISORS:

PROF. DR. KATJA ICKSTADT, DR. HENRIKE WEINERT,
YASSINE TALLEB, CARMEN VAN MEEGEN

DATE: FEBRUARY 9, 2026

Contents

1	Introduction	1
1.1	Objective	1
1.2	Research Questions	2
1.3	Approach to Problem Solving	2
1.4	Key Results	2
1.5	Report Structure	3
2	The Problem Description	3
2.1	Data Quality and Technical Constraints	3
2.2	Scope of the Analysis	4
3	Statistical Methods	5
3.1	Experimental Design	5
3.2	Genre Selection and Data Sources	5
3.3	Data Collection Protocol	5
3.4	Mathematical Framework: Cosine Similarity	6
3.5	Hypothesis Testing and Inferential Statistics	7
3.5.1	Impact of the Initial Playlist	7
3.5.2	Impact of User Demographics	8
3.6	Resources and the Use of Artificial Intelligence	8
4	Statistical Analysis	9
4.1	Exploratory Data Analysis	9
4.2	Impact of Initial Playlist	12
4.3	Influence of User Demographics	13
4.3.1	Gender Effect Analysis	13
4.3.2	Age Cohort Effect Analysis	14
4.4	Summary of Statistical Analysis	15
5	Summary	15
5.1	Final Remarks	15
A	The Code Implementation	19
A.1	Data Loading and Pre-processing	19
A.2	Data Exploration and Quality Assessment	20
A.3	Feature Extraction and User Profiling	22
A.4	Research Question 1: Impact of Initial Playlist	22
A.5	Research Question 2: Influence of User Demographics	25
A.6	Data Collection and API Integration	26

1 Introduction

The music industry has shifted from physical ownership to streaming-based access, with Spotify currently leading the market with over 600 million monthly active users. Spotify’s algorithmic curation, which aims to create a personalized listening experience through features such as “Discover Weekly” and “Daily Mix” is critical to its success. However, these systems frequently function as black boxes, with the mechanism controlling song selection hidden from the user. The lack of transparency has led to a growing corpus of study into algorithmic bias and the “Personalization-Diversity Trade-off”, which investigates how platforms combine a user’s known preferences with the need for content variety (Anderson et al., 2020).

In the field of Music Information Retrieval (MIR), recommendation strategies are generally categorized into two pillars: collaborative filtering (CF), which leverages similar users’ behavior, and content-based filtering (CBF), which analyzes the acoustic properties of the audio signal itself (Celma, 2010). Although CF is very effective for popularity-driven discovery, it might suffer from the cold-start problem, where new users or niche tracks lack sufficient interaction data to be appropriately recommended. To address this, hybrid models have become the industry norm, with deep learning used to extract latent features from audio recordings. These properties, including rhythmic patterns and harmonic structures, enable the system to map songs into a high-dimensional vector space where similarity is a function of mathematical distance (IBM, 2026).

This study contributes to the field by investigating the relationship between the seed songs and the recommended songs in a cold-start scenario. Specifically, we analyze the initial-interaction phase of a session to determine how Spotify’s algorithm extrapolates long-term user preferences from a minimal input of only the first two songs. Furthermore, this research explores the intersection of sequential modeling and algorithmic fairness by assessing whether these early-session predictions exhibit significant variance across demographic profiles, such as age and gender.

1.1 Objective

The primary objective of this paper is to analyze the sensitivity of Spotify’s recommendation engine to both content-based seeds and user-specific metadata. We aim to measure the association between an initial two-song seed playlist and the 50 recommendations generated using the Spotify API. By simulating 90 unique user profiles, we aim to assess whether the algorithm prioritizes the initial seed playlist’s songs properties or integrates demographic data to customize the result.

1.2 Research Questions

To achieve this objective, we formulated two primary research questions that guide our statistical analysis:

- **Impact of Initial Playlist:** To what extent does the genre and the specific audio features (e.g., danceability, energy, acousticness) of the seed playlist dictate the characteristics of the recommended tracks?
- **Impact of User Demographics:** Do demographic factors—specifically age (represented by three generational cohorts) and gender—exert a statistically significant influence on the recommendations, or does the algorithm remain demographic-agnostic in its immediate response to seed songs?

1.3 Approach to Problem Solving

Our methodology utilizes a controlled simulation of 90 Spotify accounts, structured via a full factorial experimental design. To ensure a representative sample, each account was assigned a specific combination of gender (Male, Female) and an age cohort (birth years 1965, 1985, and 2005), paired with one of five primary musical genres: Pop & Mainstream; Urban & Contemporary; Electronic & Beat-Based; Rock & Heavy; and Roots, Jazz & Classical Traditions.

Each account was seeded with a two-song playlist specifically curated to contain one recent hit and one less-popular track from the early 2000s. Using the Spotipy library (Lamere, 2014) to interface with the Spotify Web API and the ReccoBeats API (ReccoBeats, 2025), we automated the retrieval of 50 recommended tracks and their audio features for each user profile.

The evaluation involves extracting high-dimensional audio feature vectors such as valence, energy, and liveness for both the seeds and the recommended songs. We employ Cosine Similarity to measure the distance between these vectors, allowing us to quantify the acoustic distance between what the user provided and what the algorithm returned (IBM, 2026). The statistical significance of these relationships is then assessed using one-tailed t-tests for genre similarity, and two-sample t-tests and One-Way ANOVA for demographic comparisons, implemented via the SciPy and Scikit-learn frameworks (Virtanen et al., 2020; Pedregosa et al., 2011).

1.4 Key Results

Our findings indicate a robust seed effect, where the genre of the initial playlist significantly predicts the audio features of the recommendations across four out of five tested genres. This confirms that for “cold-start” or new-session scenarios, Spotify relies

heavily on content-based similarity. Regarding demographic influence, the statistical analysis reveals that age and gender do not have a significant impact on the audio features of the recommendations within this experimental scope. This suggests that for short-term session-based recommendations, Spotify’s algorithm may prioritize the immediate state (the seed songs) over the user’s permanent demographic profile.

1.5 Report Structure

The remainder of this report is organized as follows: Section 2 (Problem Description) provides a detailed description of the problem, data quality, and the technical context, including challenges regarding the deprecation of certain API features. Section 3 (Statistical Methods) outlines our experimental design, data collection procedures, and statistical methods. Section 4 (Statistical Analysis) presents the comprehensive statistical analysis and visualization of results using Seaborn (Waskom, 2021). Finally, Section 5 (Summary) discusses the limitations of our study and proposes directions for future research.

2 The Problem Description

The primary problem addressed in this paper is a lack of transparency in music recommendation algorithms, particularly within the Spotify ecosystem. While Spotify offers a highly customized experience to over 600 million users, the science behind why specific songs are chosen remains unclear. As observed in recent work, these systems frequently function as black boxes, where the relationship between input (user data and listening history) and output (recommendations) is not made clear for the researcher (Anderson et al., 2020).

Our goal is to examine how two specific types of input (initial musical seeds and user demographics) influence these recommendations. Specifically, we aim to determine if Spotify’s algorithm prioritizes the acoustic properties of the songs (content-based similarity) or if it incorporates demographic variables such as age and gender (collaborative filtering/profiling) in a cold-start scenario.

2.1 Data Quality and Technical Constraints

The integrity of the statistical analysis was maintained through a detailed preprocessing workflow designed to minimize data noise and ensure consistency across the 89 validated user profiles. While the experimental design initially targeted 90 profiles, one profile was excluded during the cleaning phase to ensure a strict one-to-one mapping between unique user IDs and their assigned demographic variables. Furthermore, 273 observations were removed from the dataset in instances where the Spotify API

failed to return the entire set of acoustic metadata, ensuring that the remaining data used for vectorization was complete. Rather than implementing traditional predictive modeling or regression, we opted for a comparative statistical framework by utilizing one-sample t-tests, two-sample t-tests, and ANOVA to focus on the divergence of group means across genres and demographics.

The acoustic features utilized in this study (e.g., valence, energy, and liveness) are not subjective human labels but are generated via machine learning and digital signal processing (Anderson et al., 2020). Consequently, the analysis inherits any inherent algorithmic noise or classification variance present in Spotify’s underlying models. Within this computational context, we specifically excluded the variables “Mode” and “Key” from our similarity vectors. While these are primary musical descriptors, their categorical and integer-based scales (ranging from 0 to 1 for mode and -1 to 11 for key) do not align with the continuous, normalized distributions of the other nine dimensions. Including them would have disproportionately weighted the cosine similarity calculations and disrupted the geometric interpretability of the audio profiles, thus their exclusion ensures a more balanced and acoustically accurate comparison.

A significant technical challenge in this problem is the quantification of music similarity. To analyze music statistically, we rely on audio features (mathematical representations of a song’s audio features provided by the Spotify API). However, the deprecation of the Audio Features API posed a significant challenge for our project. This change reduces our capacity to audit algorithmic behavior. Furthermore, the Spotify API does not provide direct genre tags for individual tracks. While artist-level genres exist, many artists lack these tags entirely in the current API version, complicating the process of validating genre consistency in the recommendations.

2.2 Scope of the Analysis

In light of the previously mentioned data constraints, this study is structured as a comparative inferential analysis rather than a predictive modeling task. The primary objective is to evaluate the degree of demographic agnosticism inherent in the Spotify recommendation engine during the user’s cold-start phase. Specifically, we investigate whether the system prioritizes metadata-driven personalization, such as gender identity and age group, or if it follows strictly to a content-based filtering approach.

If the algorithm is essentially genre-driven, the acoustic vectors of the recommendations should demonstrate a high degree of cosine similarity with the initial seed tracks, remaining statistically invariant across diverse user profiles. By comparing the recommendation outputs generated for different age cohorts under identical seeding conditions, we can determine how much demographic variables influence the initial state of the recommendation latent space. This enables us to determine whether the

musical signal of the seed tracks effectively outperforms user metadata at the start of the user journey.

3 Statistical Methods

3.1 Experimental Design

To investigate the influence of user demographics and musical seeds on Spotify’s recommendations, we employed a full factorial design. This approach allowed us to systematically observe how multiple independent variables interact with one another. The experimental factors and their respective levels are detailed in Table 1.

Table 1: Full Factorial Experimental Design Factors

Factor	Levels	Count
Gender	Male, Female	2
Age Cohort	1965, 1985, 2005	3
Genre	Pop & Mainstream, Urban & Contemporary, Electronic, Rock & Heavy, Roots/Jazz/Classical	5
Total Combinations	$(2 \times 3 \times 5)$	30
Replications	3 per combination	3
Total Target Profiles		90

This design results in 30 unique experimental combinations. To ensure statistical reliability and account for algorithmic variance, each combination was replicated three times, leading to an initial target of 90 simulated user accounts and 4500 recommended songs.

3.2 Genre Selection and Data Sources

To ensure a representative sample of the musical landscape available on Spotify, five distinct genre categories were selected: Electronic & Beat-Based, Pop & Mainstream, Rock & Heavy, Roots, Jazz & Classical Traditions, and Urban & Contemporary. These categories and their constituent sub-genres were identified using the genre taxonomy provided by Chosic Chosic (2024). This source was utilized to ensure that the seed tracks for each category were acoustically representative of their respective genres, providing a standardized baseline for the content-based similarity analysis.

3.3 Data Collection Protocol

The data collection was executed using a multi-stage pipeline involving the `Spotipy` Python library to interface with the Spotify Web API for behavioral data retrieval,

followed by the Reccobeats API for acoustic feature extraction. The process followed a three-step protocol for each account to maintain experimental integrity:

1. **Account Initialization:** Simulated accounts were configured with specific birth-dates and genders according to the 2×3 factorial design. To ensure a true “cold-start” environment, no prior behavioral data existed on the accounts before the seed interaction.
2. **Seed Song Selection and Interaction:** Each account was seeded with a curated two-track sequence representing one of the five target genres. This dual-seed strategy provided the algorithm with a balanced signal:
 - **Song 1 (Mainstream):** A contemporary high-popularity track (e.g., “As It Was” by Harry Styles) to establish a baseline in the current popular music space.
 - **Song 2 (Niche/Legacy):** A lower-popularity track from the early 2000s (e.g., “Breathe on Me” by Britney Spears) to provide a deeper acoustic signal.
3. **Retrieval and Feature Extraction:** Following the playback of the seed tracks, the 50 most recently played songs were retrieved using the `user-read-recently-played` endpoint. Because the Spotify history endpoint was deprecated, the dataset was enriched via the Reccobeats API. This secondary pipeline mapped each retrieved Track ID to its high-dimensional acoustic vector (e.g., valence, instrumentality, energy), providing the quantitative basis for the statistical analysis.

3.4 Mathematical Framework: Cosine Similarity

To quantify the acoustic relationship between the initial seed playlists and the resulting algorithmic recommendations, we utilized cosine similarity, a robust metric in high-dimensional vector space models for measuring the orientation of two vectors independently of their magnitude (Manning et al., 2008). In the context of Music Information Retrieval (MIR), this metric effectively represents the closeness of recommendations to the user’s starting point within a latent audio space.

Let $\mathbf{S}$ be the average feature vector of the two seed songs and $\mathbf{R}$ be the feature vector of a recommended track. The similarity is defined as the normalized dot product of the vectors:

$$\text{Similarity}(\mathbf{S}, \mathbf{R}) = \frac{\mathbf{S} \cdot \mathbf{R}}{\|\mathbf{S}\| \|\mathbf{R}\|} = \frac{\sum_{i=1}^n S_i R_i}{\sqrt{\sum_{i=1}^n S_i^2} \sqrt{\sum_{i=1}^n R_i^2}} \quad (1)$$

The vector space was defined by nine continuous, normalized audio features: *danceability*, *energy*, *loudness*, *tempo*, *speechiness*, *acousticness*, *instrumentalness*, *liveness*, and *valence*.

Additionally, Key and Mode features were excluded from the similarity calculation. Since Key is represented by discrete integers (ranging from -1 to 11) and Mode is a binary indicator (0 or 1), they do not represent linear magnitude in the same geometric sense as the other features. Including these variables would introduce significant mathematical noise and distort the cosine distance, potentially leading to an inaccurate representation of acoustic similarity (IBM, 2026).

Furthermore, to ensure that features with larger absolute ranges (such as loudness or tempo) did not disproportionately influence the similarity score, all remaining variables were normalized to a consistent scale of $[0, 1]$ prior to vector calculation.

3.5 Hypothesis Testing and Inferential Statistics

The statistical analysis was structured to isolate the effects of the initial playlist genre (Content Based Filtering) from collaborative and demographic filtering (Age and Gender). All tests were conducted at a significance level of $\alpha = 0.05$.

3.5.1 Impact of the Initial Playlist

To determine if the recommendation engine effectively preserves the acoustic profile of the seed tracks, we conducted a series of one-sample, one-tailed t-tests. This approach evaluates whether the similarity generated by the algorithm is significantly greater than what would be expected by chance or unrelated content. The test was performed on the mean Cosine Similarity score for each of the five genres variables.

- **Hypotheses:**

- $H_0 : \mu_{similarity} \leq 0$: The recommendations are acoustically orthogonal to or different from the seeds.
- $H_1 : \mu_{similarity} > 0$: The recommendations exhibit a statistically significant positive acoustic similarity to the seeds.

- **Interpretation:** A significant result would indicate that the recommendation algorithm is effectively utilizing the acoustic features of the seed songs to generate similar content, confirming a content-based filtering effect in the cold-start scenario.

3.5.2 Impact of User Demographics

To investigate if Spotify’s cold-start recommendations are moderated by user profiling, we tested the influence of gender and age on the similarity scores. This allows us to see if the algorithm prioritizes demographic metadata over acoustic consistency.

Gender Effect Analysis: A two-sample independent t-test was employed to compare the mean similarity scores between Male (n_1) and Female (n_2) profiles.

- **Hypotheses:** $H_0 : \mu_{male} = \mu_{female}$ vs. $H_1 : \mu_{male} \neq \mu_{female}$.
- **Interpretation:** A statistically significant result ($p < 0.05$) would suggest that the recommendation algorithm treats users differently based on gender, potentially prioritizing acoustic similarity more heavily for one demographic over the other.

Age Cohort Effect Analysis: Since the age variable consists of three distinct cohorts (1965, 1985, and 2005), a one-way Analysis of Variance (ANOVA) was required.

- **Hypotheses:**
 - $H_0 : \mu_{1965} = \mu_{1985} = \mu_{2005}$ (Mean similarity scores are equal across all age groups).
 - $H_1 : \exists \mu_i \neq \mu_j$ (At least one age group’s mean similarity score differs significantly from the others).
- **Interpretation:** A significant F-statistic would indicate that the “cold-start” recommendations are moderated by the user’s age, suggesting that demographic targeting influences the acoustic consistency of the generated playlists.

3.6 Resources and the Use of Artificial Intelligence

We implemented the computational analysis for this study using the Python programming language (v3.10), utilizing specialized libraries for data manipulation, statistical inference, and visualization. We conducted data preprocessing and structured management using the Pandas library (pandas development team, 2020), while we performed numerical operations via NumPy (Harris et al., 2020). For the inferential statistical tests, including the one-sample t-tests, independent t-tests, and the one-way ANOVA, we utilized the SciPy stats module (Virtanen et al., 2020). Additionally, we employed the Seaborn library (Waskom, 2021) to generate the statistical visualizations required to interpret correlation of the song features. These tools provided the necessary framework for us to evaluate the significance of acoustic similarity across different user profiles and genres.

In addition to these analytical tools, we employed the Gemini large language model (Google) as a collaborative assistant during the preparation of this report (Google, 2024). Specifically, we used the AI for language refinement, optimizing the clarity of technical prose, conducting preliminary source searches for statistical benchmarks, and ensuring the structural integrity of the LaTeX formatting. We emphasize that all core research questions, experimental design, data collection, and the interpretation of results remain the original work of the author. The AI served strictly as an adaptive tool for editorial enhancement and technical organization, ensuring that our presentation of the findings meets academic standards without substituting for the author’s primary intellectual contribution.

4 Statistical Analysis

4.1 Exploratory Data Analysis

Before performing inferential tests, we analyzed the acoustic characteristics of the five genre categories in our seed lists. Since all audio features were normalized to a $[0, 1]$ scale, we can directly compare the mean values across genres to identify the distinct audio profiles that the recommendation algorithm used as input.

Although the experimental design initially targeted a perfectly balanced distribution, the final dataset after API cleaning and the removal of incomplete interaction logs consisted of 89 accounts. This led to 4571 rows of recommended song data, which was reduced to 4298 rows after removing 273 rows of data with missing audio features. Taking into account that some songs were recommended by the algorithm more than once, this population generated a total of 682 unique song observations.

Table 2 summarizes the mean values for the nine normalized audio features calculated across each genre. These values represent the average acoustic profile of the tracks generated by the recommendation engine for each category based on the provided seeds.

Table 2: Mean Normalized Audio Features by Genre

Genre	Acc.	Dan.	Ene.	Ins.	Liv.	Lou.	Spe.	Tem.	Val.
Pop & Mainstream	0.173	0.627	0.650	0.007	0.206	0.902	0.105	0.694	0.634
Urban & Contemporary	0.020	0.751	0.706	0.000	0.031	0.866	0.126	0.727	0.920
Electronic & Beat-Based	0.297	0.691	0.789	0.443	0.246	0.861	0.104	0.613	0.217
Rock & Heavy	0.001	0.694	0.630	0.001	0.125	0.939	0.085	0.644	0.728
Roots, Jazz & Classical	0.818	0.588	0.245	0.000	0.113	0.728	0.075	0.616	0.673

The descriptive data presented in Table 2 reveals distinct acoustic signatures for each genre, providing a baseline for evaluating the recommendation engine’s performance. Most notably, the *Roots, Jazz & Classical* category is characterized by a high acousticness score (0.818) and the lowest mean energy (0.245). In contrast, the *Elec-*

tronic & Beat-Based genre exhibits the highest energy (0.789) and a significant degree of instrumentality (0.443).

We must emphasize that these genre profiles were generated from extremely sparse seeds; due to both metadata availability within the Spotify API and the number of songs in our seeds, the input for *Urban & Contemporary* and *Roots, Jazz & Classical* consisted of only a single track ($N = 1$), while the remaining genres were seeded with only two tracks ($N = 2$). The extreme mean values observed, such as the lack of instrumentality in the Urban category or the near-zero energy in Roots, are direct consequences of this limited dimensionality.

However, the fact that the engine successfully produced 50 recommendations that maintain these specific acoustic traits from such a minimal starting point is a significant finding. It demonstrates that the algorithm does not require a large training set from the user to pivot its output. Instead, it performs a content-based recommendation, mirroring the small but effective structure of a single track across the entire discovery set rather than reverting to a more generic average of top hits. This sensitivity ensures that even in a total cold-start scenario with minimal user input, the discovery mechanism remains highly responsive to the specific genre indicated.

To examine the dependence structure of our features and visualize the data spread, we generated a correlation matrix and a box plot of the normalized values.

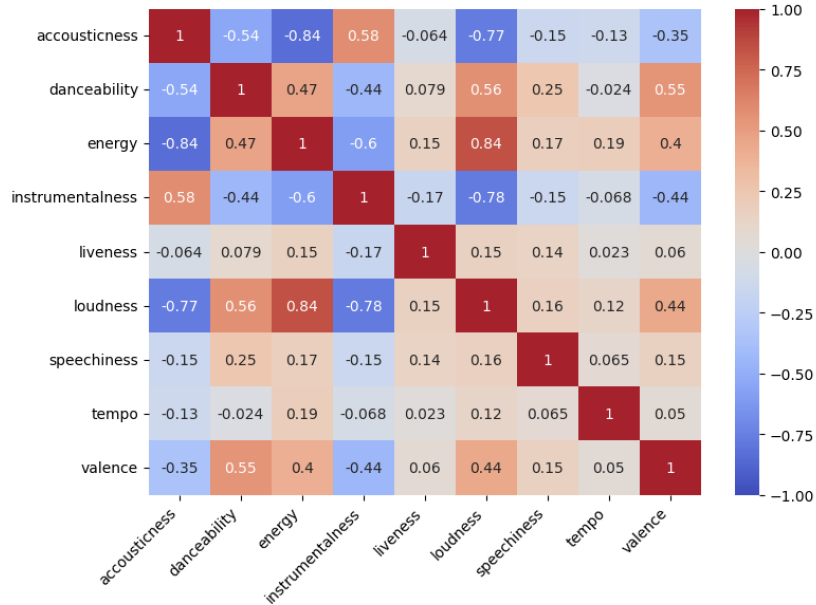


Figure 1: Correlation Matrix of Audio Features

The heatmap in Figure 1 depicts the inter-relationships between the nine acoustic dimensions, providing important context for similarity calculations. We found strong correlations between loudness and energy ($r=0.84$) and acousticness and energy ($r=-0.84$). These relationships indicate that as a track’s perceived intensity increases, its

acoustic profile tends to decrease. While these coefficients are significant, the absence of extreme multicollinearity across the entire feature set (most variables have unique variance) supports the decision to include all nine dimensions in the cosine similarity vector. This ensures that the similarity score captures a multidimensional characteristic without being redundant.

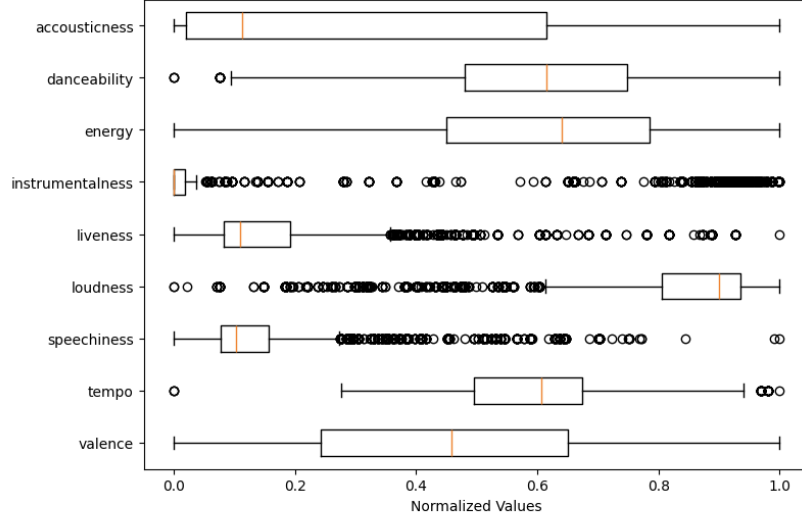


Figure 2: Distribution of Normalized Audio Features across the Dataset

As shown in Figure 2, the normalization process effectively compressed the diverse scales of the Spotify API metadata into a uniform $[0,1]$ range. The distribution of these values provides further insight into the nature of the recommendation output. For instance, loudness exhibits a higher median and a tighter interquartile range, suggesting a consistent level of dynamic compression across the recommended tracks. In contrast, acousticness exhibits a significantly lower median and a wider interquartile range, indicating that while the majority of recommendations are non-acoustic, the engine maintains the flexibility to suggest highly organic content when triggered by specific genre seeds.

The instrumentalness dimension is particularly revealing; it remains near zero for the vast majority of the dataset, reflecting a systemic algorithmic preference for less acoustic music in general discovery. However, the presence of a distinct cluster of outliers and a secondary *Electronic & Beat-Based* profile (as noted in Table 2) demonstrates that the engine can effectively pivot toward instrumental content when the input genre explicitly demands it. This confirms that while the average recommendation is non-instrumental, the content-based filtering mechanism is sensitive enough to override this vocal-led bias for specific musical contexts.

4.2 Impact of Initial Playlist

The first research question investigated whether Spotify’s recommendations significantly reflect the acoustic profile of the seed tracks. As demonstrated in Table 3, the one-sample t-tests indicate a highly significant positive cosine similarity across all genres ($p < 0.001$).

Table 3: One-Sample T-Test Results for Cosine Similarity by Genre

Genre	Mean Score	Std. Dev	n	t-statistic	p-value	Reject H_0
Electronic & Beat-Based	0.968	0.010	18	404.25	$p < .001$	True
Pop & Mainstream	0.991	0.004	17	1121.32	$p < .001$	True
Rock & Heavy	0.985	0.002	18	2115.36	$p < .001$	True
Roots, Jazz & Classical	0.852	0.045	18	80.17	$p < .001$	True
Urban & Contemporary	0.962	0.005	18	859.00	$p < .001$	True
TOTAL	0.951	0.055	89	162.08	$p < .001$	True

The results in Table 3 indicate an overwhelming rejection of the null hypothesis across all categories. The aggregate mean cosine similarity of 0.951 signifies a near-perfect alignment between the seed tracks and the resulting recommendations. Interestingly, *Pop & Mainstream* achieved the highest similarity (0.991), likely due to the highly standardized production formulas used in mainstream music. *Roots, Jazz & Classical Traditions* showed the lowest similarity (0.852) and the highest standard deviation (0.045), suggesting that the algorithm encounters more acoustic diversity when recommending within traditional genres compared to beat-based ones.

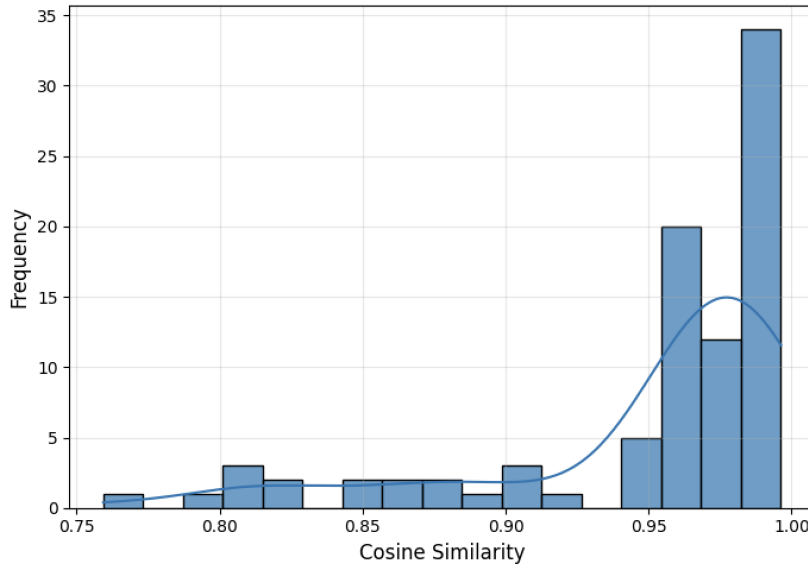


Figure 3: Histogram of Cosine Similarity

Figure 3 illustrates this high precision. The distribution is heavily left-skewed, with the vast majority of similarity scores clustering between 0.90 and 1.0. This visual evidence supports the idea that the recommendation engine is performing high-accuracy content matching.

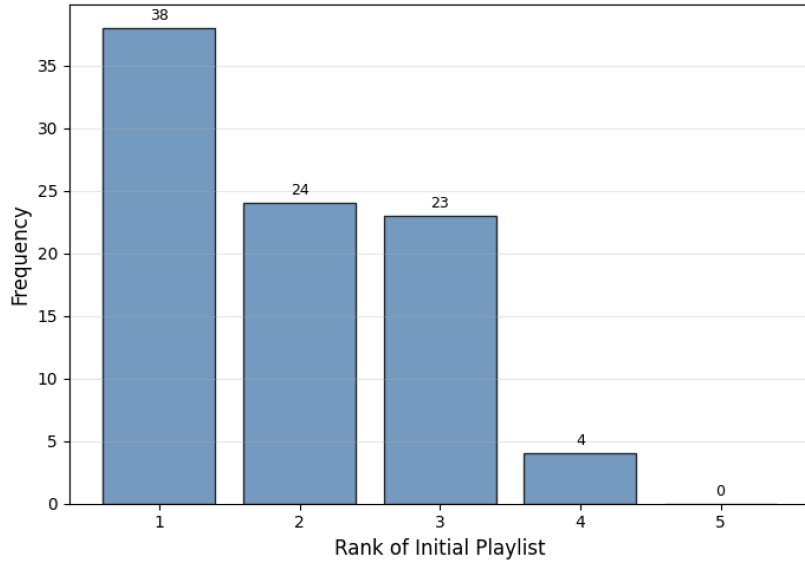


Figure 4: Rank Frequency of Seed Genre Similarity Among All Genres

Finally, Figure 4 provides a crucial ranking analysis. In nearly every instance, the seed genre’s acoustic profile was the most similar to the recommended output compared to the four alternative genres. This “Rank 1” dominance demonstrates that the algorithm’s content-based filtering is robust enough to maintain genre purity even before any user interaction (likes, skips) has occurred. This finding strongly supports the conclusion that Spotify’s recommendation system is primarily driven by the acoustic features of the seed songs in a cold-start scenario.

4.3 Influence of User Demographics

In the second phase of our inferential analysis, attention was shifted from acoustic content to user attributes. We tested whether the recommendation engine applies personalization or exhibits algorithmic bias with respect to account metadata (gender and age cohorts) in a cold-start scenario. To assess this, we compared the distributions of cosine similarity scores for recommended tracks across gender and age groups.

4.3.1 Gender Effect Analysis

To determine if gender identity influences the acoustic profile of recommendations, we conducted independent two-sample t-tests comparing Male (M) and Female (F) profiles for each genre category. The null hypothesis (H_0) posited that there is no significant difference in the mean cosine similarity between genders.

Table 4: Two-Sample T-Test Results: Gender Comparison

Genre	Mean (F)	Mean (M)	p-value	Reject H_0
Electronic & Beat-Based	0.965	0.972	0.147	False
Pop & Mainstream	0.990	0.992	0.201	False
Rock & Heavy	0.985	0.985	0.352	False
Roots, Jazz & Classical	0.848	0.855	0.751	False
Urban & Contemporary	0.963	0.962	0.438	False
TOTAL	0.950	0.953	0.785	False

The statistical results presented in Table 4 reveal a consistent failure to reject the null hypothesis across all tested genres. The p-values range from 0.147 to 0.751, all of which significantly exceed the $\alpha = 0.05$ threshold. Even in the aggregate “TOTAL” category, the difference in mean similarity between Female (0.950) and Male (0.953) profiles was statistically negligible ($t = -0.274$, $p = 0.785$).

This lack of significance suggests that the recommendation algorithm is demographically balanced at the start of a user’s journey. Rather than relying on gender-based stereotypes to fill a data void, the system appears to prioritize the acoustic properties of the seed tracks. This indicates that the content-based filtering mechanism is the primary driver of the initial experience, ensuring that recommendations remain gender-neutral before individual listening history begins to influence the model.

4.3.2 Age Cohort Effect Analysis

We further examined the role of age by categorizing users into three distinct birth-year cohorts (1965, 1985, and 2005). A One-Way ANOVA was utilized to identify if any variance in similarity scores could be attributed to the user’s age group.

Table 5: ANOVA Results: Age Cohort Comparison (1965, 1985, 2005)

Genre Group	Age Group	Mean	F-stat	p-value	Sig.
Electronic	20/40/60	0.97/0.97/0.96	0.843	0.450	No
Pop	20/40/60	0.99/0.98/0.99	1.370	0.286	No
Rock	20/40/60	0.98/0.98/0.98	0.178	0.839	No
Roots/Jazz	20/40/60	0.85/0.85/0.83	0.334	0.722	No
Urban	20/40/60	0.96/0.96/0.96	1.020	0.384	No
TOTAL	All Ages	0.95/0.95/0.94	0.108	0.898	No

As detailed in Table 5, the ANOVA results confirm that age is not a significant predictor of recommendation similarity ($p > .05$ in all cases). The total F-statistic of 0.108 ($p = .898$) indicates that the variance within each age group is far greater than the variance between them. This finding is particularly noteworthy, as it suggests that Spotify does not differentiate between its initial recommendations therefore a 60 year

old user and a 20 year old user starting with the same Electronic playlist will receive acoustically near or identical suggestions.

4.4 Summary of Statistical Analysis

The statistical evaluation of the recommendation engine reveals two important findings. First, the algorithm uses a clear seed-based recommendation logic. Despite the extreme sparsity of the seed data, where inputs were limited to only one ($N = 1$) or two tracks ($N = 2$), the engine successfully mirrored the specific acoustic vector of the seed rather than reverting to a generic popularity average. This confirms that the system is highly sensitive to initial musical signals, even when metadata is minimal.

Second, the results demonstrate a clear demographic neutrality in the cold-start phase. The consistent failure to reject the null hypothesis across all t-tests and ANOVA gender/age cohorts ($p > 0.05$) indicates that the platform does not employ demographic stereotyping at the beginning of the user journey. The acoustic output remains statistically identical regardless of the user’s age or gender, suggesting an objective function optimized for genre-consistency over demographic-based personalization.

5 Summary

This study provides empirical evidence regarding the prioritization of content-based signals within the discovery architecture during the cold-start phase. Our findings suggest that in the absence of a robust user profile, the recommendation engine defaults to an strong content-matching process. By recommending according to the profile of small seed sets, the algorithm maintains acoustic precision across diverse musical genres, effectively bypassing the data void that typically characterizes new account creation.

Furthermore, the analysis establishes a state of demographic parity within the initial recommendation cycle. The lack of statistical variance across diverse age and gender profiles indicates that the platform’s discovery logic is architecturally neutral before behavioral data is introduced. Rather than relying on demographic-based collaborative filtering to predict user preference, the engine’s primary objective function is optimized for acoustic similarity. Consequently, this research concludes that the initial cold-start experience is driven by the immediate musical context provided by the user, ensuring a genre-consistent environment that remains independent of account metadata.

5.1 Final Remarks

The results of this study highlight a robust but starting-list sensitive approach by the Spotify recommendation engine during the initial cold-start phase, which is apparently

not affected by the gender or the age group of the user. For the user, this indicates that the immediate musical context of their first playlist choice is the single most influential factor in shaping their initial algorithmic experience. Even a single seed track is enough to calibrate the engine’s output, effectively bypassing demographic stereotypes in favor of acoustic relevance.

Lastly, future research could expand upon these findings by diversifying the seed data coverage. Specifically, increasing the volume of initial seed tracks beyond the current $N = 1$ and $N = 2$ constraints would allow for a more granular analysis of how the engine handles multi-track inputs versus singular acoustic signals. Furthermore, future audits should incorporate automated genre extraction through the Spotify API’s broader metadata endpoints; this would enable a deeper investigation into whether the Spotify’s recommendation engine’s behavior is driven purely by acoustic features or by additional categorical labels assigned to the tracks. By tracking the evolution of these recommendations as more tracks are added to the initial session, we can more accurately map the transition from raw content-matching to personalized collaborative filtering.

References

- Ashton Anderson, Lucas Maystre, Ian Anderson, Rishabh Mehrotra, and Mounia Lalmas. Algorithmic effects on the diversity of consumption on spotify. In *Proceedings of The Web Conference 2020*, pages 2155–2165, 2020.
- Oscar Celma. *Music Recommendation And Discovery In The Long Tail*. Springer Science & Business Media, 2010.
- Chosic. List of music genres: Comprehensive guide and examples, 2024. URL <https://www.chosic.com/list-of-music-genres/>. Accessed: 2024-05-20.
- Google. Gemini 1.5 flash: A large language model for multimodal reasoning and assistance, 2024. URL <https://deepmind.google/technologies/gemini/>.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Pícus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2. URL <https://doi.org/10.1038/s41586-020-2649-2>.
- IBM. What is cosine similarity?, 2026. URL <https://www.ibm.com/think/topics/cosine-similarity>. Accessed: 2026-02-04.
- Paul Lamere. *Welcome to Spotipy! — spotipy 2.0 documentation*, 2014. URL <https://spotipy.readthedocs.io/en/2.25.1/>. Retrieved October 26, 2025.
- Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- The pandas development team. pandas-dev/pandas: Pandas, February 2020. URL <https://doi.org/10.5281/zenodo.3509138>.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- ReccoBeats. Music recommendation and database api service, 2025. URL <https://reccobeats.com/>. Accessed: 2025-10-26.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

Michael L. Waskom. Seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021. doi: 10.21105/joss.03021.

A The Code Implementation

A.1 Data Loading and Pre-processing

The following Python script handles the initial data ingestion, cleaning of user demographic inconsistencies, and the normalization of audio features using the `MinMaxScaler`.

Listing 1: Data Loading and Pre-processing Script

```
1 import ast
2 import json
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6 import seaborn as sns
7 from collections import Counter
8 from scipy import stats
9 from sklearn.metrics.pairwise import cosine_similarity
10 from sklearn.preprocessing import StandardScaler, MinMaxScaler
11
12 # Global display configurations
13 np.set_printoptions(precision=3)
14 pd.set_option('display.max_columns', None)
15 pd.set_option('display.max_colwidth', None)
16 pd.set_option('display.float_format', '{:.3f}'.format)
17 pd.set_option('display.precision', 3)
18 pd.set_option('display.expand_frame_repr', False)
19
20 # Path to the dataset
21 path = 'played_out.csv'
22 data = pd.read_csv(path)
23
24 # Genre mapping based on experimental seed playlists
25 genre_dict = {
26     1: {'name': 'Pop & Mainstream', 'init_playlist': ['4
    ↪ Dvkj6JhhA12EX05fT7y2e', '1lK5iIMKifrxERzS3iimJH']},
27     2: {'name': 'Urban & Contemporary', 'init_playlist': ['27
    ↪ NovPIUIR0ZoCHxABJwK', '6nz35DNIzbtj5ztpDEcW1j']},
28     3: {'name': 'Electronic & Beat-Based', 'init_playlist': ['4
    ↪ uUG5RXr0k84mYEFvj3cK', '7p6oXzBSPAXXz8Xb8gBPki']},
29     4: {'name': 'Rock & Heavy', 'init_playlist': ['4pt5fDVTg5GhEvEtlz9dKk'
    ↪ , '3ENHpbTuY72FukZbwGP6bc']},
```

```

30     5: {'name': 'Roots, Jazz & Classical Traditions', 'init_playlist': ['0
    ↪ G9qcE5Gz6xTz8jWXlRgBx', '40gRh2ZURv5tYBo8yB9Lu3']}
31 }
32
33 audio_features = ['acousticness', 'danceability', 'energy', '
    ↪ instrumentalness',
34                   'liveness', 'loudness', 'speechiness', 'tempo', 'valence
    ↪ ']
35
36 # Data Cleaning: Remove redundant columns and parse metadata
37 data = data.drop(columns=['Unnamed: 0', 'track.is_local', 'track.type'])
38 data['track.artists'] = data['track.artists'].apply(lambda x: ast.
    ↪ literal_eval(x)[0]['name'])
39 data['first_genre'] = data['first_genre'].apply(lambda x: genre_dict.get(x
    ↪ )['name'])
40
41 # Resolve demographic inconsistencies by selecting the most frequent entry
    ↪ per user
42 counts = data.groupby(['user', 'gender', 'age']).size().reset_index(name='
    ↪ count')
43 most_common = counts.sort_values(['user', 'count'], ascending=[True, False
    ↪ ]).drop_duplicates('user')
44 data = data.merge(most_common[['user', 'gender', 'age']], on=['user', '
    ↪ gender', 'age'], how='inner')
45
46 # Temporal sorting
47 data = data.sort_values(by=['user', 'played_at']).reset_index(drop=True)
48
49 # Handling missing values
50 data_no_miss = data.dropna()
51
52 # Feature Normalization (Min-Max Scaling to [0, 1])
53 scaler = MinMaxScaler()
54 scaled_data = data.copy()
55 scaled_data[audio_features] = scaler.fit_transform(data[audio_features])

```

A.2 Data Exploration and Quality Assessment

The following script performs a diagnostic check on the user base, identifies missing values across columns, and visualizes the relationship between acoustic features using a correlation heatmap and a normalized boxplot.

Listing 2: Data Diagnostics and Initial Visualization

```

1  # User and data volume statistics
2  user_count = Counter(data['user'])
3  print(f'Number of users -> {len(user_count)}')
4  print(f'Number of songs per user -> min = {np.min(list(user_count.values()
    ↪ ))}, mean = {np.mean(list(user_count.values()))}, max = {np.max(
    ↪ list(user_count.values()))}')
5
6  # Missing value analysis
7  missing_data = data[data.isna().any(axis=1)]
8  missing_counts = missing_data.isna().sum()
9  columns_with_missing = missing_counts[missing_counts > 0]
10 print("\nColumns with missing values:")
11 print(columns_with_missing)
12
13 # Visualization of feature distributions and correlations
14 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
15
16 # Subplot 1: Correlation Matrix
17 ax1 = axes[0]
18 corr_matrix = data[audio_features].corr()
19 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, vmin=-1,
    ↪ vmax=1, ax=ax1)
20 ax1.set_xticklabels(ax1.get_xticklabels(), rotation=45, ha='right')
21
22 # Subplot 2: Boxplot of Normalized Audio Features
23 ax2 = axes[1]
24 reversed_audio_features = audio_features[::-1]
25 data_np = scaled_data[reversed_audio_features].dropna().values
26 x_min, x_max = np.min(data_np), np.max(data_np)
27 x_padding = 0.05 * (x_max - x_min)
28 x_min, x_max = x_min - x_padding, x_max + x_padding
29 ax2.boxplot(scaled_data[reversed_audio_features].dropna().values, vert=
    ↪ False, tick_labels=reversed_audio_features)
30 ax2.set_xlabel('Normalized Values')
31 ax2.set_xlim([x_min, x_max])
32
33 plt.tight_layout()
34 plt.show()

```

A.3 Feature Extraction and User Profiling

This segment extracts unique track profiles and aggregates audio features at the genre level (based on seed tracks) and the user level. It specifically filters out the initial interaction period to establish a clean user profile.

Listing 3: Track, Genre, and User Feature Profiling

```
1 # Extracting distinct track signatures
2 distinct_track = scaled_data[['track.id', 'track.name'] + audio_features].
    ↪ drop_duplicates()
3
4 # Aggregating audio features for the seed genre definitions
5 distinct_genre = pd.DataFrame(columns=['genre'] + audio_features)
6 for k, v in genre_dict.items():
7     # Summing features of tracks within the initial seed playlists
8     row = [v['name']] + distinct_track[distinct_track['track.id'].isin(v['
    ↪ init_playlist'])][audio_features].sum().tolist()
9     distinct_genre.loc[len(distinct_genre)] = row
10    genre_dict[k]['audio_features'] = row[1:]
11
12 # Generating distinct user profiles based on post-initial interaction data
13 # Filtering ensures profiles are built from discovery content rather than
    ↪ seeds
14 user_dict = {}
15 distinct_user = scaled_data[~scaled_data['played_at'].str.startswith('
    ↪ 2025-10-1')]
16 distinct_user = distinct_user.groupby(['user', 'gender', 'age']).agg({
17     **{col: 'mean' for col in audio_features},
18     'first_genre': lambda x: x.mode()[0]
19 }).reset_index()[['user', 'gender', 'age', 'first_genre'] + audio_features
    ↪ ]
20
21 # Mapping user data to a dictionary for downstream statistical modeling
22 for index, row in distinct_user.iterrows():
23     user_dict[row['user']] = row[2:].values.tolist()
```

A.4 Research Question 1: Impact of Initial Playlist

This section implements the Cosine Similarity model to measure the distance between the recommended content and the initial seed genres. It further calculates one-tailed t-tests to evaluate the statistical significance of the content-matching accuracy.

Listing 4: Cosine Similarity Matrix and Genre Ranking

```

1  # Calculate Cosine Similarity between user recommendations and genre seeds
2  similarity_df = distinct_user.copy()
3  similarity_df['recommended_audio_features'] = similarity_df[audio_features
    ↪ ].apply(lambda x: np.array(x), axis=1)
4  similarity_df = similarity_df.drop(columns=audio_features)
5
6  for k, v in genre_dict.items():
7      genre_vec = np.array(genre_dict[k]['audio_features']).reshape(1, -1)
8      # Applying cosine similarity for each genre vector
9      similarity_df[v['name']] = similarity_df['recommended_audio_features'
    ↪ ].apply(
10         lambda x: cosine_similarity(x.reshape(1, -1), genre_vec)[0][0]
11     )
12
13 # Determine the rank of the initial seed genre within the recommendations
14 similarity_df['first_genre_score'] = similarity_df.apply(lambda row: row[
    ↪ row['first_genre']], axis=1)
15 similarity_df['recommended_genre'] = similarity_df.apply(
16     lambda row: sorted(distinct_genre['genre'].tolist(), key=lambda g: row
    ↪ [g], reverse=True), axis=1
17 )
18 similarity_df['first_genre_rank'] = similarity_df.apply(
19     lambda row: row['recommended_genre'].index(row['first_genre']) + 1,
    ↪ axis=1
20 )

```

Listing 5: Statistical Significance Testing (RQ1)

```

1  # Define 1-sample t-test function for genre similarity
2  def calculate_ttest(genre, df):
3      data = df['first_genre_score'].tolist() if genre == 'TOTAL' else df[df
    ↪ ['first_genre'] == genre]['first_genre_score'].tolist()
4      # Test if the mean similarity is significantly greater than 0
5      return stats.ttest_1samp(data, popmean=0, alternative='greater')
6
7  # Aggregate statistics by genre
8  genre_stats = similarity_df.groupby('first_genre')['first_genre_score'].
    ↪ agg(['mean', 'std', 'count'])
9  overall_mean = similarity_df['first_genre_score'].mean()
10 overall_std = similarity_df['first_genre_score'].std()
11 overall_count = len(similarity_df)

```

```

12
13 result = genre_stats.rename(columns={'mean': 'mean_score', 'std': 'std_dev
    ↪ ', 'count': 'n_samples'})
14 result.loc['TOTAL'] = [overall_mean, overall_std, overall_count]
15 result = result.reset_index()
16
17 # Perform the t-test and evaluate H0 rejection
18 result['one-tailed t_test H0: mu <= 0'] = result['first_genre'].apply(
    ↪ lambda x: calculate_ttest(x, similarity_df))
19 result['reject_H0 (alpha = 0.05)'] = result['one-tailed t_test H0: mu <= 0
    ↪ '].apply(lambda x: x[1] < 0.05)

```

Listing 6: Visualizing Content Fidelity and Rank Distribution

```

1 # Plotting the distribution of initial playlist ranks and similarity
    ↪ scores
2 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
3
4 # Subplot 1: Rank Frequency (Evaluation of Genre-Consistency)
5 ax1 = axes[0]
6 rank_counts = similarity_df['first_genre_rank'].value_counts().sort_index
    ↪ ()
7 all_ranks = range(1, len(distinct_genre['genre']) + 1)
8 values = [rank_counts.get(rank, 0) for rank in all_ranks]
9 bars = ax1.bar(all_ranks, values, width=0.8, edgecolor='black', alpha=0.8,
    ↪ color='steelblue')
10 ax1.set_xlabel('Rank of Initial Playlist', fontsize=12)
11 ax1.set_ylabel('Frequency', fontsize=12)
12 ax1.set_xticks(all_ranks)
13
14 # Subplot 2: Histogram and KDE of Cosine Similarity Scores
15 ax2 = axes[1]
16 sns.histplot(data=similarity_df, x='first_genre_score', kde=True, alpha
    ↪ =0.7, ax=ax2)
17 ax2.set_xlabel('Cosine Similarity', fontsize=12)
18 ax2.set_ylabel('Frequency', fontsize=12)
19
20 plt.tight_layout()
21 plt.show()

```

A.5 Research Question 2: Influence of User Demographics

The following scripts evaluate whether demographic variables (Gender and Age) significantly influence the similarity scores of the recommended tracks.

Listing 7: Gender-Based Demographic Analysis (Independent Samples t-test)

```
1 # Function to calculate independent samples t-test between Female (F) and
  ↪ Male (M) cohorts
2 def calculate_ttest_ind(genre, df):
3     if genre == 'TOTAL':
4         data1 = df[df['gender'] == 'F']['first_genre_score'].tolist()
5         data2 = df[df['gender'] == 'M']['first_genre_score'].tolist()
6     else:
7         data1 = df[(df['first_genre'] == genre) & (df['gender'] == 'F')]['
  ↪ first_genre_score'].tolist()
8         data2 = df[(df['first_genre'] == genre) & (df['gender'] == 'M')]['
  ↪ first_genre_score'].tolist()
9     return stats.ttest_ind(data1, data2)
10
11 # Aggregating metrics by genre and gender
12 result_gender = similarity_df.groupby(['first_genre', 'gender']).agg({'
  ↪ first_genre_score': ['mean', 'std', 'count']}).reset_index()
13 result_gender.columns = ['first_genre', 'gender', 'mean_score', 'std_dev',
  ↪ 'n_samples']
14
15 # Appending global TOTALS for gender cohorts
16 for g in ['M', 'F']:
17     subset = similarity_df[similarity_df['gender'] == g]['
  ↪ first_genre_score']
18     result_gender.loc[len(result_gender)] = ['TOTAL', g, subset.mean(),
  ↪ subset.std(), len(subset)]
19
20 # Statistical evaluation of H0: No difference between gender means
21 result_gender['t_test H0: mu1 = mu2'] = result_gender.apply(
22     lambda row: calculate_ttest_ind(row['first_genre'], similarity_df),
  ↪ axis=1
23 )
24 result_gender['reject_H0 (alpha = 0.05)'] = result_gender['t_test H0: mu1
  ↪ = mu2'].apply(lambda x: x[1] < 0.05)
```

Listing 8: Age-Based Demographic Analysis (One-way ANOVA)

```

1 # Function to calculate One-way ANOVA across three age cohorts (20, 40,
  ↪ 60)
2 def calculate_anova(genre, df):
3     groups = [20, 40, 60]
4     if genre == 'TOTAL':
5         data_groups = [df[df['age'] == a]['first_genre_score'].tolist()
  ↪ for a in groups]
6     else:
7         data_groups = [df[(df['first_genre'] == genre) & (df['age'] == a)
  ↪ ]['first_genre_score'].tolist() for a in groups]
8     return stats.f_oneway(*data_groups)
9
10 # Aggregating metrics by genre and age group
11 result_age = similarity_df.groupby(['first_genre', 'age']).agg({
  ↪ 'first_genre_score': ['mean', 'std', 'count']}).reset_index()
12 result_age.columns = ['first_genre', 'age', 'mean_score', 'std_dev', '
  ↪ n_samples']
13
14 # Appending global TOTALS for age cohorts
15 for a in [20, 40, 60]:
16     subset = similarity_df[similarity_df['age'] == a]['first_genre_score']
17     result_age.loc[len(result_age)] = ['TOTAL', a, subset.mean(), subset.
  ↪ std(), len(subset)]
18
19 # Statistical evaluation of H0: Equality of means across all age groups
20 result_age['ANOVA H0: mu1 = mu2 = mu3'] = result_age.apply(
21     lambda row: calculate_anova(row['first_genre'], similarity_df), axis=1
22 )
23 result_age['reject_H0 (alpha = 0.05)'] = result_age['ANOVA H0: mu1 = mu2 =
  ↪ mu3'].apply(lambda x: x[1] < 0.05)
24
25 # LaTeX Table Export Logic
26 for row in result_age.itertuples():
27     print(f'{row.first_genre} & {row.age} & {row.mean_score:.3f} & {row.
  ↪ std_dev:.3f} & {row.n_samples} & {row._6[0]:.3f} & {row._6[1]:.3f}
  ↪ & {"No" if (row._7 is False) else "Yes"} \\\\'

```

A.6 Data Collection and API Integration

The following script utilizes the `Spotipy` library to authenticate via OAuth 2.0 and retrieve the “Recently Played” history. This script was used across all demographic

accounts to generate the raw dataset for analysis.

Listing 9: Spotify API Data Retrieval Script

```
1 import spotipy
2 from spotipy.oauth2 import SpotifyOAuth
3 import re
4 import pandas as pd
5
6 # User Configuration
7 user_name = "User_ID_Placeholder"
8 genre = "Roots, Jazz & Classical Traditions"
9 gender = "M"
10 age = 60
11
12 def read_client_credentials(credential_path: str) -> tuple[str, str, str]
13     ↪ | None:
14     try:
15         with open(credential_path) as credentials:
16             lines = credentials.read().splitlines()
17             check_format_of_credentials(lines)
18             return lines[0], lines[1], lines[2]
19     except ValueError as ve:
20         print(ve)
21
22 def check_format_of_credentials(lines) -> ValueError | None:
23     if len(lines) != 3:
24         raise ValueError("Credentials file incomplete.")
25     if len(lines[0]) != 32 or len(lines[1]) != 32:
26         raise ValueError("Client ID or Secret incorrectly specified.")
27     # URI Regex validation
28     pattern = re.compile(r"^https?:/(?:[A-Za-z0-9-]+\.)+[A-Za-z]{2,}(?:\
29     ↪ d+)?(?:/.*)?$")
30     if not pattern.fullmatch(lines[2]):
31         raise ValueError("Invalid redirect URI.")
32
33 # Authentication and Scope Definition
34 client_credential_path = "credentials.txt"
35 scope = "user-read-recently-played"
36
37 # Initialize the Spotipy object
38 sp = spotipy.Spotify(auth_manager=SpotifyOAuth(
39     scope=scope,
```

```
38     client_id=client_id,
39     client_secret=client_secret,
40     redirect_uri=redirect_uri))
41
42 # Retrieve the 50 most recently played tracks
43 played = sp.current_user_recently_played()
44
45 # Flatten JSON response and append demographic metadata
46 played_df = pd.json_normalize(played["items"])
47 played_df["first_genre"] = genre
48 played_df["user"] = f"{client_id}_{user_name}"
49 played_df["gender"] = gender
50 played_df["age"] = age
51
52 # Export to CSV for individual account tracking
53 played_df.to_csv(f"played_{client_id}.csv", header=True)
```